SYSTEM SYGNALIZACJI ON-AIR

DOKUMENTACJA TECHNICZNA I ARCHITEKTURA OPROGRAMOWANIA

Sterownik kasetonu studyjnego sprzężony ze środowiskiem REAPER DAW

INSTRUKCJA PROGRAMISTY (IT DEVELOPER MANUAL)

Szczegółowy opis implementacji firmware ESP32, protokołu MQTT, skryptu ReaScript oraz pełne listingi kodu źródłowego

Autor opracowania:	Zespół Inżynierii Oprogramowania Systemów Wbudowanych

Wersja dokumentacji:	2.0 (Wydanie produkcyjne po refaktoryzacji)
Mikrokontroler:	Espressif ESP32-WROOM-32 (Dual-Core 240 MHz, 4 MB Flash)
Środowisko kompilacji:	PlatformIO Core v6.x / Framework Arduino ESP32
Języki oprogramowania:	C++11 (ESP32), Python 3 (ReaScript), Lua 5.3 (Autostart)
Protokół sieciowy:	MQTT v3.1.1 (QoS 1, retain flag) over TCP/IP
Data publikacji:	Październik 2026

Spis treści

1 Architektura systemu i protokół komunikacji **2**
1.1 Topologia i przepływ danych w systemie . . . 2
1.2 Specyfikacja protokołu MQTT . . . 2

2 Warstwa sprzętowa sterownika ESP32 **3**
2.1 Przypisanie linii wejścia/wyjścia (GPIO) . . . 3
2.2 Uzasadnienie przeniesienia diody statusowej z GPIO 5 na GPIO 21 . . . 3
2.3 Polaryzacja wyjścia przekaźnikowego . . . 4

3 Szczegółowa analiza implementacji firmware ESP32 **5**
3.1 Architektura obiektowa: Klasa `OnAirController` . . . 5
3.1.1 Główne pola stanu i mechanizmy zarządzania . . . 5
3.2 Eliminacja przerwań sprzętowych ISR na korzyść bezpiecznego debouncingu . . . 5
3.3 Filtracja styków przycisku Factory Reset . . . 5
3.4 Integracja z WiFiManager i obsługa automatycznego restartu . . . 6
3.5 Optymalizacja parsera MQTT (Zero-Allocation Parser) . . . 6
3.6 Maszyna stanów wskaźnika LED bez użycia biblioteki `Ticker` . . . 6

4 Szczegółowa analiza integracji po stronie REAPER DAW **7**
4.1 Skrypt główny: `onAirSystem.py` . . . 7
4.1.1 Zarządzanie połączeniem MQTT i wątkowość . . . 7
4.1.2 Mechanizm synchronizacji i callback `on_connect` . . . 7
4.1.3 Cykliczne sprawdzanie stanu transportu (`checkRecordState`) . . . 7
4.2 Skrypt startowy: `__startup.lua` . . . 7

5 Środowisko deweloperskie i procedura kompilacji **9**
5.1 Konfiguracja projektu: `platformio.ini` . . . 9
5.2 Procedura kompilacji i wgrywania oprogramowania . . . 9

6 Pełne listingi kodów źródłowych **10**
6.1 Konfiguracja środowiska: `platformio.ini` . . . 10
6.2 Firmware mikrokontrolera ESP32: `src/main.cpp` . . . 11
6.3 Integracja z programem REAPER: `Reaper/onAirSystem.py` . . . 19
6.4 Skrypt automatycznego startu: `Reaper/__startup.lua` . . . 21

1 Architektura systemu i protokół komunikacji

System sygnalizacji ON-AIR został zaprojektowany w trójwarstwowej architekturze rozproszonej, w której stacja robocza audio (DAW), pośrednik komunikatów (Broker MQTT) oraz fizyczny kontroler wykonawczy (ESP32) współpracują bezprzewodowo w czasie rzeczywistym.

1.1 Topologia i przepływ danych w systemie

Struktura przepływu sygnałów pomiędzy komponentami prezentuje się następująco:

[**REAPER DAW (Stacja Robocza)**]
⇓ *ReaScript API: odpytywanie stanu polecenia Record (Command ID: 1013)*
[**Skrypt integracyjny: `onAirSystem.py` (Python 3 / Paho-MQTT)**]
⇓ *Publikacja stanu: Topic `studio/onair`, Payload: `"ON"` / `"OFF"`, QoS=1, Retain=True*
[**Broker MQTT (np. Mosquitto / EMQX / Home Assistant)**]
⇓ *Dystrybucja wiadomości przez Wi-Fi 802.11 b/g/n (Port 1883)*
[**Sterownik ESP32 (`PubSubClient3` + `OnAirController`)**]
⇓ *Aktualizacja wyjść cyfrowych GPIO (Przekaźnik SSR/EM: GPIO 4, Dioda aktywności: GPIO 18)*
[**Kaseton Ostrzegawczy ON-AIR (Lampa 230V AC / LED 12V DC)**]

1.2 Specyfikacja protokołu MQTT

Komunikacja pomiędzy oprogramowaniem nadrzędnym a węzłem wykonawczym odbywa się zgodnie ze standardem MQTT v3.1.1:

- **Domyślny temat (Topic):** `studio/onair` (z możliwością rekonfiguracji z poziomu portalu WWW oraz skryptu REAPERa).
- **Format ładunku (Payload):** Prosty ciąg tekstowy ASCII:
  - `"ON"` lub `"1"` – Rozpoczęto rejestrację audio (załączenie lampy).
  - `"OFF"` lub `"0"` – Zatrzymano rejestrację audio (wyłączenie lampy).
- **Flaga zachowania stanu (`Retain Flag = True`):** Publikacja komunikatów ze strony REAPERa odbywa się z włączoną flagą `retain=True`. Oznacza to, że broker MQTT zapamiętuje ostatni znany stan w danym temacie. Gdy sterownik ESP32 uruchamia się po zaniku zasilania lub powraca z trybu manualnego, natychmiast po subskrypcji tematu otrzymuje aktualny stan nagrywania bez konieczności oczekiwania na kolejne kliknięcie przycisku *Record* w REAPERze.
- **Poziom jakości usług (`QoS = 1`):** Gwarantuje dostarczenie wiadomości przynajmniej raz (*At least once*) wraz z potwierdzeniem odbioru (`PUBACK`) ze strony brokera.
- **Identyfikacja klienta (Client ID):** Każdy egzemplarz urządzenia generuje unikalny, stały identyfikator klienta oparty na 48-bitowym sprzętowym adresie MAC z pamięci OTP eFuse ESP32 (format: `OnAir-XXXXXXXXXXXX`). Zapobiega to kolizjom połączeń na brokerze i nie tworzy osieroconych sesji w przypadku ponawiania połączenia.

2 Warstwa sprzętowa sterownika ESP32

2.1 Przypisanie linii wejścia/wyjścia (GPIO)

Projekt wykorzystuje mikrokontroler ESP32-WROOM-32. Wszystkie piny zostały dobrane ze szczególnym uwzględnieniem specyfiki układu ESP32 (linie strappingowe, taktowanie szyn, domyślne stany rozruchowe).

Tabela 1: Konfiguracja wyprowadzeń mikrokontrolera ESP32

Stała w kodzie	**GPIO**	**Kierunek**	**Funkcja / Uzasadnienie techniczne**
`PIN_RELAY`	GPIO 4	OUTPUT	Sterowanie wejściem modułu przekaźnika (SSR / elektromagnetycznego) lampy ON-AIR. Pin bezpieczny rozruchowo, brak stanów nieustalonych (PWM) przy starcie.
`PIN_STATUS_LED`	GPIO 21	OUTPUT	Dioda LED sygnalizacji stanu sieci Wi-Fi i brokera MQTT. Zmieniono z GPIO 5, aby wyeliminować konflikt z linią strappingową magistrali SDIO/VSPI.
`PIN_ACTIVE_LED`	GPIO 18	OUTPUT	Dioda LED sygnalizacji aktywnego stanu ON-AIR (świeci synchronicznie z załączeniem przekaźnika).
`PIN_PASSTHROUGH_SWITCH`	GPIO 19	INPUT_PULLUP	Wejście przełącznika kołyskowego trybu ręcznego (zielony klawisz). Zwarcie do masy (`LOW`) oznacza wymuszenie stanu ON i wyłączenie radia Wi-Fi.
`PIN_FACTORY_RESET`	GPIO 22	INPUT_PULLUP	Wejście przycisku zerowania konfiguracji. Zwarcie do masy przez 5 sekund uruchamia procedurę kasowania pamięci NVS.

2.2 Uzasadnienie przeniesienia diody statusowej z GPIO 5 na GPIO 21

W pierwotnej wersji projektu dioda statusowa była podłączona do pinu **GPIO 5**. Na układach ESP32 pin ten pełni krytyczną funkcję strappingową:

- Podczas rozruchu (*Reset/Power-On*) układ próbkujący odczytuje poziom logiczny na GPIO 5 w celu ustalenia timingu magistrali SPI oraz trybu pracy kontrolera pamięci Flash (*SDIO Slave timing*).
- Standardowe podłączenie diody LED z rezystorem ograniczającym do masy (GND) wymusza na pinie stan logiczny niski (`LOW`) w chwili restartu (domyślnie pin ten wymaga stanu `HIGH` przez wewnętrzny rezystor podciągający).

- W zależności od partii produkcyjnej i wersji bootloadera może to skutkować niestabilnością fazy startowej lub błędami komunikacji z pamięcią Flash.
- Przeniesienie wskaźnika na **GPIO 21** (czysty pin ogólnego przeznaczenia bez powiązań strappingowych) całkowicie rozwiązuje ten problem i gwarantuje 100% stabilność każdego restartu.

2.3 Polaryzacja wyjścia przekaźnikowego

W kodzie zaimplementowano konfigurowalną na etapie kompilacji logikę polaryzacji wyjścia:

```
constexpr uint8_t RELAY_ACTIVE_LEVEL = HIGH;
constexpr uint8_t RELAY_INACTIVE_LEVEL = (RELAY_ACTIVE_LEVEL == HIGH) ? LOW : HIGH;
```

Umożliwia to błyskawiczną adaptację oprogramowania zarówno do modułów wyzwalanych stanem wysokim (*Active-HIGH*), jak i stanem niskim (*Active-LOW*, np. wyzwalanych przez transoptor lub przekaźniki półprzewodnikowe SSR typu Omron G3MB-202P) bez modyfikowania logiki sterującej w głębi programu.

3 Szczegółowa analiza implementacji firmware ESP32

3.1 Architektura obiektowa: Klasa `OnAirController`

Cały stan urządzenia, zarządzanie pamięcią nieulotną, obsługa sieci Wi-Fi, klienta MQTT oraz filtracji wejść cyfrowych zostały zamknięte wewnątrz klasy `OnAirController`.

3.1.1 Główne pola stanu i mechanizmy zarządzania

- **Buforowanie parametrów:** Zmienne tablicowe `mqttServer[40]`, `mqttPort[6]` oraz `mqttTopic[64]` przechowują parametry konfiguracyjne. Są one deklarowane w klasie przed obiektami `WiFiManagerParameter`, co gwarantuje poprawną kolejność inicjalizacji list konstrukcyjnych w języku C++.
- **Trwała pamięć NVS (`Preferences`):** Obsługa przestrzeni nazw `"onAirApp"`. Zapis i odczyt kluczy `"mqttIp"`, `"mqttPort"` oraz `"mqttTopic"`.
- **Metoda `saveCustomParams()`:** Parametry z portalu konfiguracyjnego są pobierane i zapisywane do NVS natychmiast po zatwierdzeniu formularza przez użytkownika (callback `setSaveParamsCallback`). Zabezpiecza to przed utratą wpisanych danych w sytuacji, gdy połączenie z routerem Wi-Fi nie nastąpiło natychmiast.

3.2 Eliminacja przerwań sprzętowych ISR na korzyść bezpiecznego debouncingu

W pierwotnej wersji kodu przełącznik Passthrough był obsługiwany za pomocą przerwania sprzętowego `attachInterrupt(..., CHANGE)`. Takie rozwiązanie niosło ze sobą poważne ryzyko awarii:

1. Podczas operacji zapisu lub kasowania pamięci Flash NVS (`preferences.putString`, `preferences.clear`) oraz operacji stosu Wi-Fi kontroler ESP32 tymczasowo wyłącza pamięć podręczną instrukcji (*SPI Flash Cache*).
2. Wystąpienie drgań styków przełącznika w tym ułamku sekundy powodowało wywołanie kodu obsługi przerwania odwołującego się do pamięci poza sekcją `IRAM_ATTR`, skutkując natychmiastowym błędem krytycznym:

```
Guru Meditation Error: Core 1 panic'ed (Cache disabled but cached memory
region accessed)
```

3. Zwykły przełącznik kołyskowy nie wymaga mikrosekundowych czasów reakcji. W obecnej wersji zaimplementowano nieblokujący filtr programowy o oknie czasowym 50 ms w metodzie `checkPassthroughSwitch()`. Zapewnia to zerowe ryzyko kolizji z operacjami NVS i całkowitą odporność na zakłócenia mechaniczne styków.

3.3 Filtracja styków przycisku Factory Reset

Metoda `monitorFactoryReset()` została wyposażona w analogiczny 50-milisekundowy filtr drgań styków. Wcześniejszy brak debouncingu powodował, że mikroskopijne przerwy stykowe podczas 5-sekundowego trzymania przycisku zerowały licznik czasu, sprawiając wrażenie, że przycisk „zawodzi lub działa losowo". Obecna implementacja gwarantuje precyzyjne odliczenie 3 sekund (aktywacja stroboskopu ostrzegawczego) oraz 5 sekund (wykonanie procedury *wipe & reboot*).

3.4 Integracja z WiFiManager i obsługa automatycznego restartu

- **Tryb nieblokujący:** Wywołanie `wifiManager.setConfigPortalBlocking(false)` pozwala na ciągłe przetwarzanie pętli głównej `runLoop()` i obsługę przełączników sprzętowych nawet wtedy, gdy aktywny jest punkt dostępowy konfiguracji.
- **Limit czasu (`Timeout = 120 s`):** Portal konfiguracyjny ma zdefiniowany limit czasu 2 minut.
- **Wstrzymywanie limitu czasu (`setAPClientCheck(true)`):** Gdy jakiekolwiek urządzenie podłączy się do sieci `OnAir_Setup`, funkcja sprawdzająca liczbę podłączonych stacji wstrzymuje odliczanie timera. Użytkownik ma nieograniczony czas na konfigurację, dopóki jest połączony z siecią AP.
- **Automatyczny restart po wygaśnięciu portalu:** W przypadku braku aktywności przez 120 sekund callback `onConfigPortalTimeout()` oraz detekcja stanu `!wifiManager.getConfigPortalActive` wykonują automatyczny restart układu (`esp_restart()`). Zapobiega to trwałemu pozostaniu sterownika w trybie AP po chwilowym zaniku zasilania routera w studiu.

3.5 Optymalizacja parsera MQTT (Zero-Allocation Parser)

W procedurze `globalMqttCallback` wyeliminowano dynamiczną alokację obiektów `String` w pętli bajt po bajcie. Nowy parser operuje bezpośrednio na surowym wskaźniku bufora `payload`:

1. Odcina wiodące i końcowe białe znaki (spacje, `\t`, `\r`, `\n`) poprzez przesuwanie wskaźnika początkowego i dekrementację licznika długości.
2. Wykonuje szybkie, bezalokacyjne porównanie pamięci bez uwzględniania wielkości liter za pomocą `strncasecmp`:
   - Długość 2 bajty i zawartość `"ON"` LUB długość 1 bajt i znak `'1'` → załączenie stanu ON-AIR.
   - Długość 3 bajty i zawartość `"OFF"` LUB długość 1 bajt i znak `'0'` → wyłączenie stanu ON-AIR.

Rozwiązanie to eliminuje fragmentację pamięci sterty (*heap fragmentation*) mikrokontrolera podczas długotrwałej pracy ciągłej i zapewnia deterministyczny czas reakcji.

3.6 Maszyna stanów wskaźnika LED bez użycia biblioteki `Ticker`

Zrezygnowano z biblioteki `Ticker`, która uruchamiała callbacki w kontekście wątku timerów programowych FreeRTOS (`esp_timer`). Wprowadzono silnie typowane wyliczenie `enum class LedState` obsługiwane deterministycznie w pętli `runLoop()` przy użyciu funkcji `millis()`:

- `OFF` – dioda wygaszona.
- `SOLID_ON` – stan wysoki na stałe.
- `FAST_BLINK` – przełączanie stanu co 200 ms (łączenie z siecią Wi-Fi / brokerem MQTT).
- `SLOW_BLINK` – przełączanie stanu co 1000 ms (aktywny portal konfiguracyjny AP).
- `STROBE_BLINK` – przełączanie stanu co 60 ms (ostrzeżenie przed resetem fabrycznym).

4 Szczegółowa analiza integracji po stronie REAPER DAW

4.1 Skrypt główny: `onAirSystem.py`

Skrypt działa w środowisku Python osadzonym w procesie programu REAPER za pośrednictwem natywnego interfejsu ReaScript API.

4.1.1 Zarządzanie połączeniem MQTT i wątkowość

- Skrypt inicjalizuje klienta `paho.mqtt.client` z jawnie określoną wersją interfejsu `CallbackAPIVersion.VERS` (zgodność z `paho-mqtt >= 2.0`).
- Wywołanie `self.mqttClient.loop_start()` uruchamia dedykowany wątek w tle odpowiedzialny za obsługę gniazda TCP, wymianę pakietów kontrolnych (*PINGREQ/PINGRESP*) oraz automatyczną rekoneksję w razie utraty połączenia z brokerem.
- Parametr `keepalive` ustalono na 60 sekund, co eliminuje ryzyko zerwania sesji podczas obciążenia procesora renderowaniem projektów audio.

4.1.2 Mechanizm synchronizacji i callback `on_connect`

W definicji klienta zaimplementowano funkcję zwrotną wywoływaną przy każdym nawiązaniu połączenia z brokerem:

```
def on_connect(client, userdata, flags, rc, properties=None):
    currentState = RPR_GetToggleCommandState(1013)
    payload = "ON" if currentState == 1 else "OFF"
    client.publish(self.topic, payload, qos=1, retain=True)
```

Gwarantuje to, że w momencie nawiązania lub wznowienia sesji z brokerem bieżący stan transportu programu REAPER jest natychmiast synchronizowany z kolejką MQTT.

4.1.3 Cykliczne sprawdzanie stanu transportu (`checkRecordState`)

Funkcja `checkRecordState()` odpytuje silnik REAPERa o stan polecenia nagrywania:

```
currentState = RPR_GetToggleCommandState(1013)
```

Wartość `1013` to stały, wbudowany identyfikator akcji REAPERa (*Transport: Record*). Jeżeli stan uległ zmianie w stosunku do poprzedniego cyklu (`self.lastState`), skrypt publikuje komunikat `"ON"` lub `"OFF"`. Następnie planuje swoje kolejne wywołanie w głównej pętli czasowej GUI REAPERa (ok. 30 Hz):

```
RPR_defer("onAirApp.checkRecordState()")
```

Dzięki wykorzystaniu `RPR_defer` zamiast pętli `while` z funkcją `sleep()`, skrypt działa asynchronicznie i nie blokuje głównego wątku interfejsu użytkownika ani wątku przetwarzania buforów audio (ASIO/WASAPI).

4.2 Skrypt startowy: `__startup.lua`

Plik `__startup.lua` umieszczony w katalogu `Scripts` programu REAPER jest automatycznie wykonywany podczas każdego uruchomienia środowiska DAW:

```
scriptId = "_RSd98776a0c7b58c3b6f282be8231148942ddfa014"
reaper.Main_OnCommand(reaper.NamedCommandLookup(scriptId), 0)
```

Wyszukuje on identyfikator zarejestrowanej akcji skryptu `onAirSystem.py` i inicjuje jego działanie w tle od razu po otwarciu programu.

Uwaga wdrożeniowa: Identyfikator akcji `_RS...` w REAPERze

ciąg znaków `_RSd98776a0c7b58c3b6f282be8231148942ddfa014` jest skrótem SHA-1 generowanym przez REAPERa na podstawie ścieżki skryptu w momencie jego ręcznego dodania do *Action List* (`New action` → `Load ReaScript`). W przypadku przenoszenia systemu na inną maszynę lub zmiany struktury katalogów należy w oknie *Action List* kliknąć prawym przyciskiem myszy na skrypt `onAirSystem.py`, wybrać **„Copy selected action command ID”** i zaktualizować zmienną `scriptId` w pliku `__startup.lua`.

5 Środowisko deweloperskie i procedura kompilacji

Projekt firmware zarządzany jest za pośrednictwem narzędzia PlatformIO Core.

5.1 Konfiguracja projektu: `platformio.ini`

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    tzapu/WiFiManager @ ^2.0.17
    hmueller01/PubSubClient3@^3.3.0
```

5.2 Procedura kompilacji i wgrywania oprogramowania

Wszystkie operacje można wykonać z poziomu terminala PlatformIO CLI:

1. **Kompilacja projektu (weryfikacja kodu):**

```
pio run
```

2. **Kompilacja i wgranie firmware przez port USB/UART:**

```
pio run --target upload
```

3. **Uruchomienie monitora diagnostycznego portu szeregowego:**

```
pio device monitor -b 115200
```

4. **Pełne czyszczenie pamięci Flash (wraz z partycją NVS i danymi Wi-Fi):**

```
pio run --target erase
```

6 Pełne listingi kodów źródłowych

W niniejszym rozdziale zawarto kompletne, produkcyjne kody źródłowe wszystkich modułów wchodzących w skład systemu sygnalizacji ON-AIR.

6.1 Konfiguracja środowiska: `platformio.ini`

Listing 1: Plik konfiguracyjny środowiska PlatformIO (`platformio.ini`)

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    tzapu/WiFiManager @ ^2.0.17
    hmueller01/PubSubClient3@^3.3.0
```

6.2 Firmware mikrokontrolera ESP32: `src/main.cpp`

Listing 2: Główny kod źródłowy oprogramowania ESP32 (`src/main.cpp`)

```cpp
#include <Arduino.h>
#include <Preferences.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include <WiFiManager.h>
#include <esp_system.h>

#define DEBUG_ONAIR 1

#if DEBUG_ONAIR
#define DEBUG_PRINT(x) Serial.print(x)
#define DEBUG_PRINTLN(x) Serial.println(x)
#else
#define DEBUG_PRINT(x)
#define DEBUG_PRINTLN(x)
#endif

// ========================================
// Hardware Configuration
// ========================================
constexpr int PIN_RELAY = 4;
constexpr int PIN_STATUS_LED = 21;
constexpr int PIN_ACTIVE_LED = 18;
constexpr int PIN_PASSTHROUGH_SWITCH = 19;
constexpr int PIN_FACTORY_RESET = 22;

// Relay output logic:
// Set to HIGH if relay energizes on HIGH (e.g. transistor switch).
// Set to LOW if relay energizes on LOW (typical for optocoupled relay modules).
constexpr uint8_t RELAY_ACTIVE_LEVEL = HIGH;
constexpr uint8_t RELAY_INACTIVE_LEVEL = (RELAY_ACTIVE_LEVEL == HIGH) ? LOW : HIGH;

// ========================================
// LED Indicator States
// ========================================
enum class LedState {
  OFF,
  SOLID_ON,      // Connected to WiFi and MQTT broker
  FAST_BLINK,    // Connecting to WiFi or MQTT broker (200ms)
  SLOW_BLINK,    // WiFi configuration portal active (1000ms)
  STROBE_BLINK   // Factory reset hold warning (60ms)
};

class OnAirController;
extern OnAirController onAirSystem;

// C-style callback wrappers for external libraries
void globalSaveParamsCallback();
void globalSaveConfigCallback();
void globalConfigModeCallback(WiFiManager *myWiFiManager);
void globalConfigPortalTimeoutCallback();
void globalMqttCallback(char *topic, byte *payload, unsigned int length);

/**
 * @class OnAirController
 * @brief Fully encapsulated controller managing WiFi, MQTT, hardware overrides,
 * non-blocking debouncing, and system state.
 */
class OnAirController {
private:
  // Network configuration buffers
  char mqttServer[40] = "";
  char mqttPort[6] = "1883";
  char mqttTopic[64] = "studio/onair";

  // System state flags
  bool lastMqttCommandState = false;
  bool isPassthroughActive = false;
  bool isNetworkSetupActive = false;
```

```
  bool isFactoryResetWarning = false;

  // Timers
  unsigned long lastMqttRetryTime = 0;
  unsigned long lastWiFiRetryTime = 0;
  unsigned long lastLedToggleTime = 0;
  bool ledCurrentLevel = false;

  // Core objects
  Preferences preferences;
  WiFiClient espClient;
  PubSubClient mqttClient;

  WiFiManager wifiManager;
  WiFiManagerParameter customMqttServer;
  WiFiManagerParameter customMqttPort;
  WiFiManagerParameter customMqttTopic;

public:
  OnAirController()
      : mqttClient(espClient),
        customMqttServer("server", "MQTT Broker IP", mqttServer, sizeof(mqttServer)),
        customMqttPort("port", "MQTT Broker Port", mqttPort, sizeof(mqttPort)),
        customMqttTopic("topic", "MQTT Topic", mqttTopic, sizeof(mqttTopic)) {
    wifiManager.addParameter(&customMqttServer);
    wifiManager.addParameter(&customMqttPort);
    wifiManager.addParameter(&customMqttTopic);
  }

  /**
   * @brief Initializes GPIO pins and initial states.
   */
  void initializeHardware() {
    pinMode(PIN_RELAY, OUTPUT);
    pinMode(PIN_STATUS_LED, OUTPUT);
    pinMode(PIN_ACTIVE_LED, OUTPUT);
    pinMode(PIN_PASSTHROUGH_SWITCH, INPUT_PULLUP);
    pinMode(PIN_FACTORY_RESET, INPUT_PULLUP);

    digitalWrite(PIN_RELAY, RELAY_INACTIVE_LEVEL);
    digitalWrite(PIN_STATUS_LED, LOW);
    digitalWrite(PIN_ACTIVE_LED, LOW);

    // Initial check of physical passthrough switch
    isPassthroughActive = (digitalRead(PIN_PASSTHROUGH_SWITCH) == LOW);
    DEBUG_PRINTLN(F("[HW] Hardware pins initialized."));
  }

  /**
   * @brief Loads MQTT Broker configuration from Non-Volatile Storage (NVS).
   */
  void loadConfig() {
    preferences.begin("onAirApp", false);
    String savedServer = preferences.getString("mqttIp", "");
    String savedPort = preferences.getString("mqttPort", "1883");
    String savedTopic = preferences.getString("mqttTopic", "studio/onair");
    preferences.end();

    if (savedServer.length() > 0) {
      strlcpy(mqttServer, savedServer.c_str(), sizeof(mqttServer));
      DEBUG_PRINT(F("[CONFIG] Loaded MQTT IP: "));
      DEBUG_PRINTLN(mqttServer);
    } else {
      DEBUG_PRINTLN(F("[CONFIG] No MQTT IP saved."));
    }

    if (savedPort.length() > 0) {
      strlcpy(mqttPort, savedPort.c_str(), sizeof(mqttPort));
    }
    if (savedTopic.length() > 0) {
      strlcpy(mqttTopic, savedTopic.c_str(), sizeof(mqttTopic));
    }

```

```
    customMqttServer.setValue(mqttServer, sizeof(mqttServer));
    customMqttPort.setValue(mqttPort, sizeof(mqttPort));
    customMqttTopic.setValue(mqttTopic, sizeof(mqttTopic));
  }

  /**
   * @brief Reads custom parameters from WiFiManager and commits them to NVS.
   */
  void saveCustomParams() {
    strlcpy(mqttServer, customMqttServer.getValue(), sizeof(mqttServer));
    strlcpy(mqttPort, customMqttPort.getValue(), sizeof(mqttPort));
    strlcpy(mqttTopic, customMqttTopic.getValue(), sizeof(mqttTopic));

    preferences.begin("onAirApp", false);
    preferences.putString("mqttIp", mqttServer);
    preferences.putString("mqttPort", mqttPort);
    preferences.putString("mqttTopic", mqttTopic);
    preferences.end();

    DEBUG_PRINTLN(F("[CONFIG] Custom parameters saved to NVS:"));
    DEBUG_PRINT(F("  Broker: "));
    DEBUG_PRINTLN(mqttServer);
    DEBUG_PRINT(F("  Port:   "));
    DEBUG_PRINTLN(mqttPort);
    DEBUG_PRINT(F("  Topic:  "));
    DEBUG_PRINTLN(mqttTopic);
  }

  /**
   * @brief Initializes WiFiManager with non-blocking AP and portal timeout.
   */
  void setupWiFiAndPortal() {
    DEBUG_PRINTLN(F("[WIFI] Initializing WiFi/Portal..."));
    WiFi.mode(WIFI_STA);
    WiFi.setAutoReconnect(true);

    wifiManager.setSaveParamsCallback(globalSaveParamsCallback);
    wifiManager.setSaveConfigCallback(globalSaveConfigCallback);
    wifiManager.setAPCallback(globalConfigModeCallback);
    wifiManager.setConfigPortalTimeoutCallback(globalConfigPortalTimeoutCallback);
    wifiManager.setConfigPortalBlocking(false);
    wifiManager.setConfigPortalTimeout(120); // 2-minute timeout for configuration portal
    wifiManager.setAPClientCheck(true);      // Keep portal alive while a client is connected to AP
    wifiManager.setWebPortalClientCheck(true); // Reset timeout on web page activity

    if (wifiManager.autoConnect("OnAir_Setup")) {
      DEBUG_PRINTLN(F("[WIFI] AutoConnect success."));
      isNetworkSetupActive = false;
      saveCustomParams();
    } else {
      DEBUG_PRINTLN(F("[WIFI] Starting AP Portal (OnAir_Setup)..."));
      isNetworkSetupActive = true;
    }
  }

  /**
   * @brief Configures MQTT client server and callback.
   */
  void setupMqtt() {
    if (isNetworkSetupActive || strlen(mqttServer) == 0) {
      return;
    }

    uint16_t port = (uint16_t)atoi(mqttPort);
    if (port == 0) {
      port = 1883;
    }

    mqttClient.setServer(mqttServer, port);
    mqttClient.setCallback(globalMqttCallback);
    DEBUG_PRINTLN(F("[MQTT] Client configured."));
  }

```

```
/**
 * @brief Non-blocking connection attempt to MQTT broker.
 */
void handleMqttConnectionNonBlocking() {
  if (isNetworkSetupActive || isPassthroughActive || WiFi.status() != WL_CONNECTED) {
    return;
  }

  if (strlen(mqttServer) == 0) {
    DEBUG_PRINTLN(F("[MQTT] Broker IP not set, skipping connection."));
    return;
  }

  uint16_t port = (uint16_t)atoi(mqttPort);
  if (port == 0) port = 1883;
  mqttClient.setServer(mqttServer, port);
  mqttClient.setCallback(globalMqttCallback);

  DEBUG_PRINT(F("[MQTT] Attempting connection to "));
  DEBUG_PRINT(mqttServer);
  DEBUG_PRINT(F(":"));
  DEBUG_PRINTLN(port);

  // Persistent client ID based on ESP32 MAC address
  char clientId[32];
  uint64_t mac = ESP.getEfuseMac();
  snprintf(clientId, sizeof(clientId), "OnAir-%04X%08X", (uint16_t)(mac >> 32), (uint32_t)mac);

  if (mqttClient.connect(clientId)) {
    DEBUG_PRINTLN(F("[MQTT] Connected to broker."));
    DEBUG_PRINT(F("[MQTT] Subscribing to: "));
    DEBUG_PRINTLN(mqttTopic);
    mqttClient.subscribe(mqttTopic);
  } else {
    DEBUG_PRINT(F("[MQTT] Connect failed, rc="));
    DEBUG_PRINTLN(mqttClient.state());
  }
}

/**
 * @brief Shuts down network stack when Passthrough mode is active.
 */
void disconnectAndSleepNetwork() {
  DEBUG_PRINTLN(F("[NET] Shutting down network services..."));
  if (isNetworkSetupActive) {
    wifiManager.stopConfigPortal();
    isNetworkSetupActive = false;
  }
  if (mqttClient.connected()) {
    mqttClient.disconnect();
  }
  WiFi.disconnect(true, false);
  WiFi.mode(WIFI_OFF);
  DEBUG_PRINTLN(F("[NET] Network offline."));
}

/**
 * @brief Updates the relay and indicator LEDs based on Passthrough or MQTT command.
 */
void updateOutputs() {
  bool finalOutputState = isPassthroughActive || lastMqttCommandState;

  DEBUG_PRINT(F("[OUTPUT] Passthrough: "));
  DEBUG_PRINT(isPassthroughActive);
  DEBUG_PRINT(F(" | MQTT: "));
  DEBUG_PRINT(lastMqttCommandState);
  DEBUG_PRINT(F(" | Final: "));
  DEBUG_PRINTLN(finalOutputState);

  digitalWrite(PIN_RELAY, finalOutputState ? RELAY_ACTIVE_LEVEL : RELAY_INACTIVE_LEVEL);
  digitalWrite(PIN_ACTIVE_LED, finalOutputState ? HIGH : LOW);
}

```

```
289  /**
290   * @brief Sets MQTT state received from broker callback.
291   */
292  void setMqttCommandState(bool state) {
293    DEBUG_PRINT(F("[MQTT] State set to: "));
294    DEBUG_PRINTLN(state ? "ON" : "OFF");
295    lastMqttCommandState = state;
296    updateOutputs();
297  }
298
299  void onConfigModeStarted() {
300    DEBUG_PRINTLN(F("[WIFI] Entered configuration mode."));
301    isNetworkSetupActive = true;
302  }
303
304  void onConfigPortalTimeout() {
305    DEBUG_PRINTLN(F("\n[WIFI] Config portal timed out! Rebooting in 500ms..."));
306    digitalWrite(PIN_STATUS_LED, LOW);
307    delay(500);
308    esp_restart();
309  }
310
311  /**
312   * @brief Checks the physical Passthrough switch with 50ms software debounce.
313   */
314  void checkPassthroughSwitch() {
315    bool reading = (digitalRead(PIN_PASSTHROUGH_SWITCH) == LOW);
316    static bool lastDebounceReading = false;
317    static unsigned long lastDebounceTime = 0;
318
319    if (reading != lastDebounceReading) {
320      lastDebounceTime = millis();
321      lastDebounceReading = reading;
322    }
323
324    if ((millis() - lastDebounceTime) > 50) {
325      if (reading != isPassthroughActive) {
326        isPassthroughActive = reading;
327        updateOutputs();
328
329        if (isPassthroughActive) {
330          DEBUG_PRINTLN(F("[SYS] Mode changed -> PASSTHROUGH (Manual Override ON)"));
331          disconnectAndSleepNetwork();
332        } else {
333          DEBUG_PRINTLN(F("[SYS] Mode changed -> NORMAL"));
334          setupWiFiAndPortal();
335          setupMqtt();
336        }
337      }
338    }
339  }
340
341  /**
342   * @brief Monitors the physical factory reset button with debounce filtering.
343   * Holding >= 3s starts strobe LED warning.
344   * Holding >= 5s executes immediate wipe & restart.
345   */
346  void monitorFactoryReset() {
347    bool rawReading = (digitalRead(PIN_FACTORY_RESET) == LOW);
348    static bool debouncedPressed = false;
349    static bool lastRawReading = false;
350    static unsigned long lastDebounceTime = 0;
351    static unsigned long resetPressStartTime = 0;
352
353    // 50ms software debounce filter to eliminate contact chatter
354    if (rawReading != lastRawReading) {
355      lastDebounceTime = millis();
356      lastRawReading = rawReading;
357    }
358
359    if ((millis() - lastDebounceTime) > 50) {
360      if (rawReading != debouncedPressed) {
361        debouncedPressed = rawReading;
```

```
        if (debouncedPressed) {
          resetPressStartTime = millis();
          DEBUG_PRINTLN(F("[SYS] Reset button PRESSED."));
        } else {
          isFactoryResetWarning = false;
          DEBUG_PRINTLN(F("[SYS] Reset button released."));
        }
      }
    }

    if (debouncedPressed) {
      unsigned long holdDuration = millis() - resetPressStartTime;

      // Visual warning after 3 seconds of hold
      if (holdDuration >= 3000 && !isFactoryResetWarning) {
        isFactoryResetWarning = true;
        DEBUG_PRINTLN(F("[SYS] Hold 2s more to execute Factory Reset..."));
      }

      // Trigger reset when held for 5 seconds
      if (holdDuration >= 5000) {
        DEBUG_PRINTLN(F("!!! [SYS] FACTORY RESET TRIGGERED !!!"));
        digitalWrite(PIN_STATUS_LED, HIGH);

        if (isNetworkSetupActive) {
          wifiManager.stopConfigPortal();
        }

        wifiManager.resetSettings();
        preferences.begin("onAirApp", false);
        preferences.clear();
        preferences.end();

        DEBUG_PRINTLN(F("[SYS] Preferences wiped. Rebooting in 500ms..."));
        delay(500);
        esp_restart();
      }
    }
  }

  /**
   * @brief Non-blocking status LED renderer without FreeRTOS timer dependencies.
   */
  void updateStatusLed() {
    if (isPassthroughActive) {
      digitalWrite(PIN_STATUS_LED, LOW);
      return;
    }

    LedState targetState;
    if (isFactoryResetWarning) {
      targetState = LedState::STROBE_BLINK;
    } else if (isNetworkSetupActive) {
      targetState = LedState::SLOW_BLINK;
    } else if (WiFi.status() != WL_CONNECTED || !mqttClient.connected()) {
      targetState = LedState::FAST_BLINK;
    } else {
      targetState = LedState::SOLID_ON;
    }

    unsigned long now = millis();
    switch (targetState) {
      case LedState::OFF:
        digitalWrite(PIN_STATUS_LED, LOW);
        break;

      case LedState::SOLID_ON:
        digitalWrite(PIN_STATUS_LED, HIGH);
        break;

      case LedState::FAST_BLINK:
        if (now - lastLedToggleTime >= 200) {
          lastLedToggleTime = now;
```

```
435           ledCurrentLevel = !ledCurrentLevel;
436           digitalWrite(PIN_STATUS_LED, ledCurrentLevel ? HIGH : LOW);
437         }
438         break;
439
440       case LedState::SLOW_BLINK:
441         if (now - lastLedToggleTime >= 1000) {
442           lastLedToggleTime = now;
443           ledCurrentLevel = !ledCurrentLevel;
444           digitalWrite(PIN_STATUS_LED, ledCurrentLevel ? HIGH : LOW);
445         }
446         break;
447
448       case LedState::STROBE_BLINK:
449         if (now - lastLedToggleTime >= 60) {
450           lastLedToggleTime = now;
451           ledCurrentLevel = !ledCurrentLevel;
452           digitalWrite(PIN_STATUS_LED, ledCurrentLevel ? HIGH : LOW);
453         }
454         break;
455     }
456   }
457
458   /**
459    * @brief Main loop worker.
460    */
461   void runLoop() {
462     // 1. Hardware checks
463     monitorFactoryReset();
464     checkPassthroughSwitch();
465     updateStatusLed();
466
467     // 2. WiFi portal processing if active
468     if (isNetworkSetupActive) {
469       wifiManager.process();
470       if (WiFi.status() == WL_CONNECTED) {
471         DEBUG_PRINTLN(F("[WIFI] Portal connected! Initializing MQTT..."));
472         isNetworkSetupActive = false;
473         saveCustomParams();
474         setupMqtt();
475       } else if (!wifiManager.getConfigPortalActive()) {
476         DEBUG_PRINTLN(F("[WIFI] Portal timed out / inactive! Rebooting in 500ms..."));
477         delay(500);
478         esp_restart();
479       }
480       return;
481     }
482
483     // 3. Normal mode network maintenance
484     if (!isPassthroughActive) {
485       unsigned long now = millis();
486
487       if (WiFi.status() != WL_CONNECTED) {
488         // WiFi dropped - attempt reconnection every 10 seconds
489         if (now - lastWiFiRetryTime > 10000) {
490           lastWiFiRetryTime = now;
491           DEBUG_PRINTLN(F("[WIFI] Connection lost. Reconnecting..."));
492           WiFi.reconnect();
493         }
494       } else {
495         // WiFi connected - maintain MQTT client
496         if (strlen(mqttServer) > 0) {
497           if (!mqttClient.connected()) {
498             if (now - lastMqttRetryTime > 5000) {
499               lastMqttRetryTime = now;
500               handleMqttConnectionNonBlocking();
501             }
502           } else {
503             mqttClient.loop();
504           }
505         }
506       }
507     }
```

```
  }
};

OnAirController onAirSystem;

// Callback implementations
void globalSaveParamsCallback() {
  onAirSystem.saveCustomParams();
}

void globalSaveConfigCallback() {
  onAirSystem.saveCustomParams();
}

void globalConfigModeCallback(WiFiManager *myWiFiManager) {
  onAirSystem.onConfigModeStarted();
}

void globalConfigPortalTimeoutCallback() {
  onAirSystem.onConfigPortalTimeout();
}

void globalMqttCallback(char *topic, byte *payload, unsigned int length) {
  // Trim leading whitespace/newlines
  while (length > 0 && (payload[0] == ' ' || payload[0] == '\r' || payload[0] == '\n')) {
    payload++;
    length--;
  }
  // Trim trailing whitespace/newlines
  while (length > 0 && (payload[length - 1] == ' ' || payload[length - 1] == '\r' || payload[length - 1] ==
      '\n')) {
    length--;
  }

  // Efficient in-place comparison without dynamic string allocation
  if ((length == 2 && strncasecmp((char *)payload, "ON", 2) == 0) ||
      (length == 1 && payload[0] == '1')) {
    onAirSystem.setMqttCommandState(true);
  } else if ((length == 3 && strncasecmp((char *)payload, "OFF", 3) == 0) ||
             (length == 1 && payload[0] == '0')) {
    onAirSystem.setMqttCommandState(false);
  } else {
    DEBUG_PRINT(F("[MQTT] Unknown command payload received, len="));
    DEBUG_PRINTLN(length);
  }
}

void setup() {
  Serial.begin(115200);
  delay(500);
  DEBUG_PRINTLN(F("\n--- SYSTEM BOOT ---"));

  onAirSystem.initializeHardware();
  onAirSystem.loadConfig();
  onAirSystem.updateOutputs();

  // Determine initial boot target
  bool isPassthrough = (digitalRead(PIN_PASSTHROUGH_SWITCH) == LOW);
  if (isPassthrough) {
    DEBUG_PRINTLN(F("[BOOT] Mode: PASSTHROUGH"));
    WiFi.mode(WIFI_OFF);
  } else {
    DEBUG_PRINTLN(F("[BOOT] Mode: NORMAL"));
    onAirSystem.setupWiFiAndPortal();
    onAirSystem.setupMqtt();
  }
}

void loop() {
  onAirSystem.runLoop();
}
```

6.3 Integracja z programem REAPER: `Reaper/onAirSystem.py`

Listing 3: Skrypt integracyjny dla REAPER DAW (`Reaper/onAirSystem.py`)

```python
import os
import json
import paho.mqtt.client as mqtt

class OnAirSystem:
    """Manages MQTT connection and REAPER recording state synchronization."""

    def __init__(self):
        resourcePath = RPR_GetResourcePath()
        scriptDir = os.path.join(resourcePath, "Scripts")
        self.configFile = os.path.join(scriptDir, "onAirConfig.json")
        self.mqttClient = None
        self.lastRecordState = -1
        self.topic = ""

    def loadConfig(self):
        """Loads configuration from JSON file if it exists."""
        if os.path.exists(self.configFile):
            try:
                with open(self.configFile, 'r') as fileObject:
                    return json.load(fileObject)
            except (ValueError, OSError):
                return None
        return None

    def saveConfig(self, configData):
        """Saves configuration to JSON file."""
        scriptDir = os.path.dirname(self.configFile)
        if not os.path.exists(scriptDir):
            os.makedirs(scriptDir, exist_ok=True)
        try:
            with open(self.configFile, 'w') as fileObject:
                json.dump(configData, fileObject, indent=4)
        except OSError as e:
            RPR_ShowConsoleMsg(f"[OnAir] Error saving config: {e}\n")

    def promptUserForConfig(self, defaultIp="192.168.0.3", defaultPort="1883", defaultTopic="studio/onair")
    :
        """Displays a REAPER dialog to gather MQTT settings."""
        dialogTitle = "OnAir MQTT Setup"
        captions = "Broker IP:,Port:,Topic:"
        defaultValues = f"{defaultIp},{defaultPort},{defaultTopic}"

        result = RPR_GetUserInputs(dialogTitle, 3, captions, defaultValues, 512)

        if result[0]:
            values = result[4].split(',')
            if len(values) == 3:
                return {
                    "brokerIp": values[0].strip(),
                    "port": values[1].strip(),
                    "topic": values[2].strip()
                }
        return None

    def connectToBroker(self, configData):
        """Attempts to connect to the MQTT broker."""
        try:
            self.topic = configData["topic"]
            self.mqttClient = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)

            def on_connect(client, userdata, flags, rc, properties=None):
                # When connected or reconnected to broker, republish current state
                # with retain=True so broker and clients always have fresh state.
                currentState = RPR_GetToggleCommandState(1013)
                payload = "ON" if currentState == 1 else "OFF"
                client.publish(self.topic, payload, qos=1, retain=True)

            self.mqttClient.on_connect = on_connect
```

```
            self.mqttClient.connect(configData["brokerIp"], int(configData["port"]), 60)
            self.mqttClient.loop_start()
            return True
        except Exception as e:
            RPR_ShowConsoleMsg(f"[OnAir] Broker connection error: {e}\n")
            return False

    def checkRecordState(self):
        """Checks REAPER record state and publishes to MQTT on change."""
        currentState = RPR_GetToggleCommandState(1013)

        if currentState != self.lastRecordState:
            self.lastRecordState = currentState
            payload = "ON" if currentState == 1 else "OFF"
            if self.mqttClient:
                self.mqttClient.publish(self.topic, payload, qos=1, retain=True)

        RPR_defer("onAirApp.checkRecordState()")

    def run(self):
        """Main execution logic for configuration and connection handling."""
        configData = self.loadConfig()
        connected = False

        while not connected:
            if not configData:
                configData = self.promptUserForConfig()
                if not configData:
                    return
                self.saveConfig(configData)

            if self.connectToBroker(configData):
                connected = True
                self.checkRecordState()
            else:
                configData = self.promptUserForConfig(
                    configData.get("brokerIp", "192.168.0.3"),
                    configData.get("port", "1883"),
                    configData.get("topic", "studio/onair")
                )
                if not configData:
                    return
                self.saveConfig(configData)


if __name__ == "__main__":
    onAirApp = OnAirSystem()
    onAirApp.run()
```

6.4 Skrypt automatycznego startu: `Reaper/__startup.lua`

Listing 4: Skrypt automatycznego uruchamiania akcji w REAPERze (`Reaper/__startup.lua`)

```
-- Remember to add onAirSystem.py script into Reaper
scriptId = "_RSd98776a0c7b58c3b6f282be8231148942ddfa014"
reaper.Main_OnCommand(reaper.NamedCommandLookup(scriptId), 0)
```